ARM PrimeCell

Synchronous Serial Port (PL022)

User Guide

ARM PrimeCell PL022 (SSP) Synchronous Serial Port User Guide (PL022 USGD 0000 A)

Open Access

ARM PrimeCell Synchronous Serial Port (PL022) User Guide

Proprietary notice

Document confidentiality status

This document is Open Access. This document has no restriction on distribution.

Product status

The information in this document is Final (information on a developed product).

Web address

http://www.arm.com

Feedback

ARM limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Feedback on this product

If you have any comments or suggestions about this product, please contact your supplier giving:

- The product name
- A concise explanation of your comments.

Contents

1 ABOUT THIS DOCUMENT

1.1 Change History

Issue	Date	Change
A	31 January, 2002	First release, supersedes PL000 USGD 0000 D generic userguide

1.2 References

For further information refer to the following documents.

Ref	Doc No	Title
1	ARM IHI 0011	AMBA Specification (Rev 2.0)
2	ARM DDI 0046	AMBA Test Interface Driver (‘TICBOX’ & TIC test methodology)
3	PL022 DDES 0000	ARM PrimeCell SSP (PL022) Design Manual

1.3 Terms and abbreviations

This document uses the following terms and abbreviations.

Term	Meaning
AHB	Advanced High-performance Bus
AMBA	Advanced Micro-controller Bus Architecture
APB	Advanced Peripheral Bus
ASB	Advanced System Bus
ASIC	Application Specific Integrated Circuit
IP	Intellectual Property
LDV	Cadence Tool Suite
RTL	Register Transfer Level
SSP	Synchronous Serial Port

2 SCOPE

This document describes the instructions for the PrimeCell Synchronous Serial Port (PL022) product.

For further information on AMBA, refer to the AMBA Specification [Ref.1].

For further information on the AMBA TIC test methodology, refer to the AMBA Test Interface Driver [Ref.2].

The ARM PrimeCell SSP (PL022) Design Manual [Ref. 3] contains details of the rtl blocks, testbench and test code descriptions.

3 DIRECTORY STRUCTURE

The peripheral development environment is stored in two directory structures, a peripheral specific directory structure and a project global directory structure. This emphasises that projects that contain multiple peripherals should be able to access the same shared data. The top level directory structures are shown in ***Figure 3-1.***

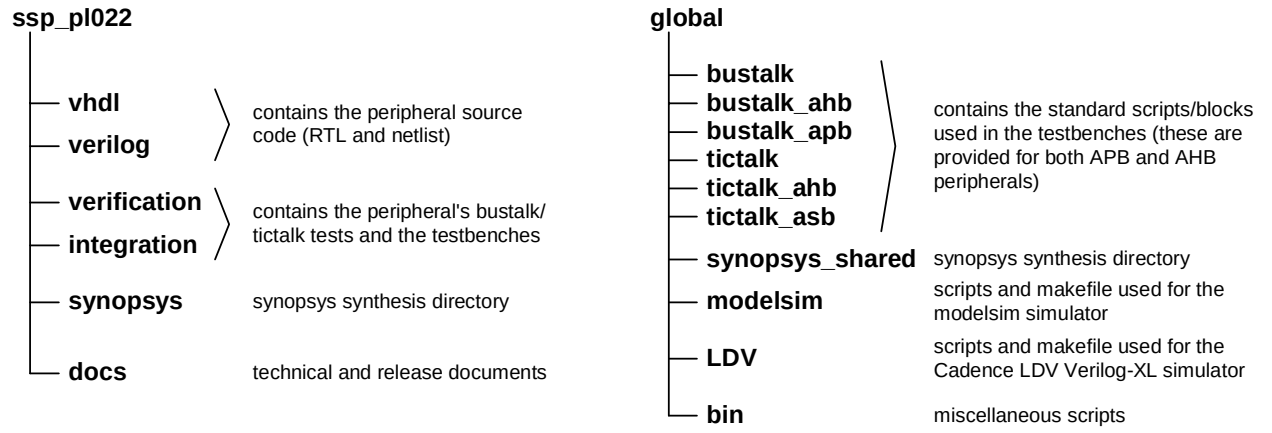
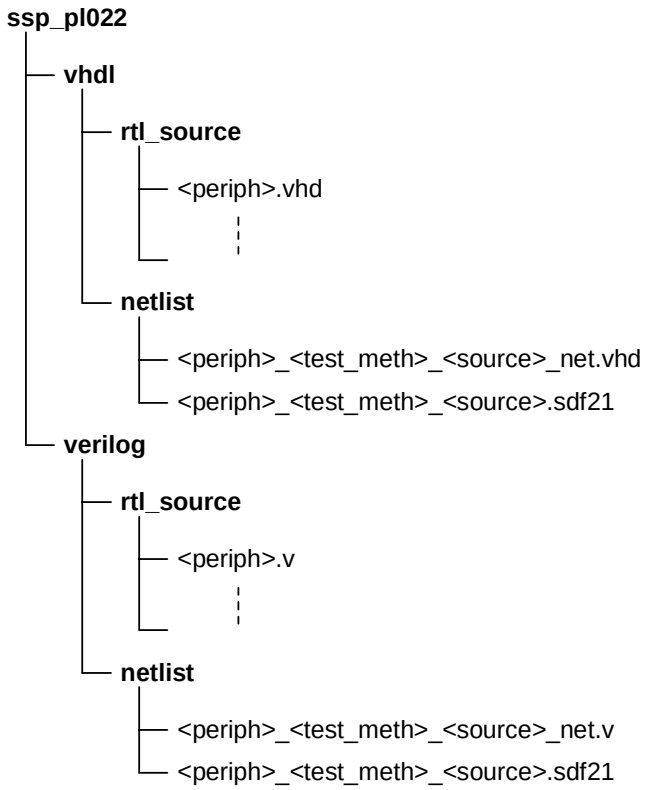


Figure 3-1. Peripheral Development Environment Directory Structure

3.1 Peripheral's source code directory structure

The peripheral's source code will be stored in two identical directory structures, as shown in ***Figure 3-2***. The source code is in the "rtl_source" directory and the synthesised netlists and associated sdf will be placed in the netlist directory.



<periph>	the name of the peripheral
<test_meth>	the test methodology: noscan, scanready, scaninsert
<source>	source code format: vhdl or verilog

Figure 3-2. Source Code Directory Structure

3.2 Generic Synopsys synthesis reference directory structure

Synthesis is supported for Synopsis Design Compiler with the required command scripts and constraint files being supplied to the customer. The synthesis environment directory structure is shown in ***Figure 3-3*** Generic Synopsys synthesis reference directory structure.

The synthesised netlist is written into the source code area's netlist.

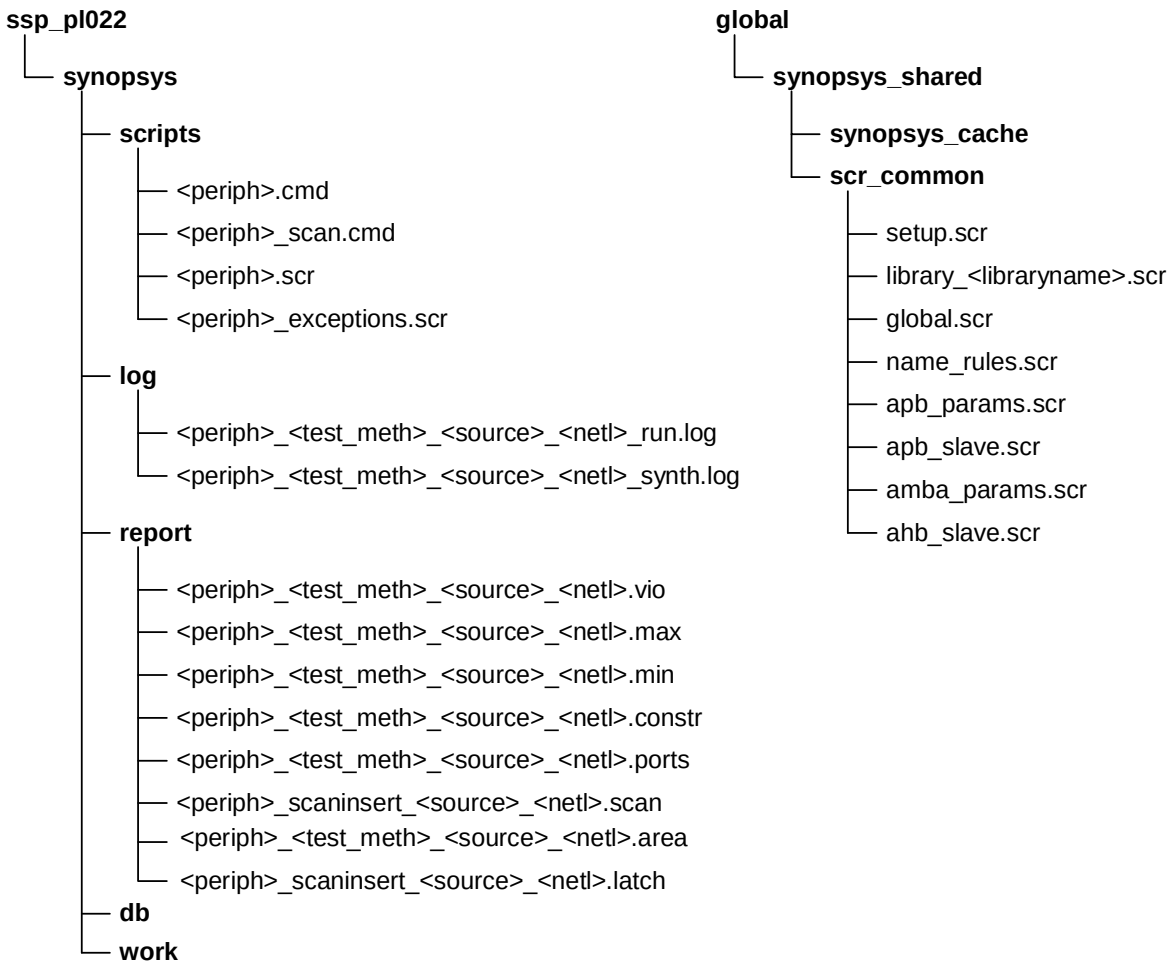


Figure 3-3. Generic Synopsys synthesis reference directory structure

3.3 Generic Verification directory structure

The verification directory contains the standalone verification testbenches in both VHDL and Verilog and the BusTalk test program, which is used for functional verification.

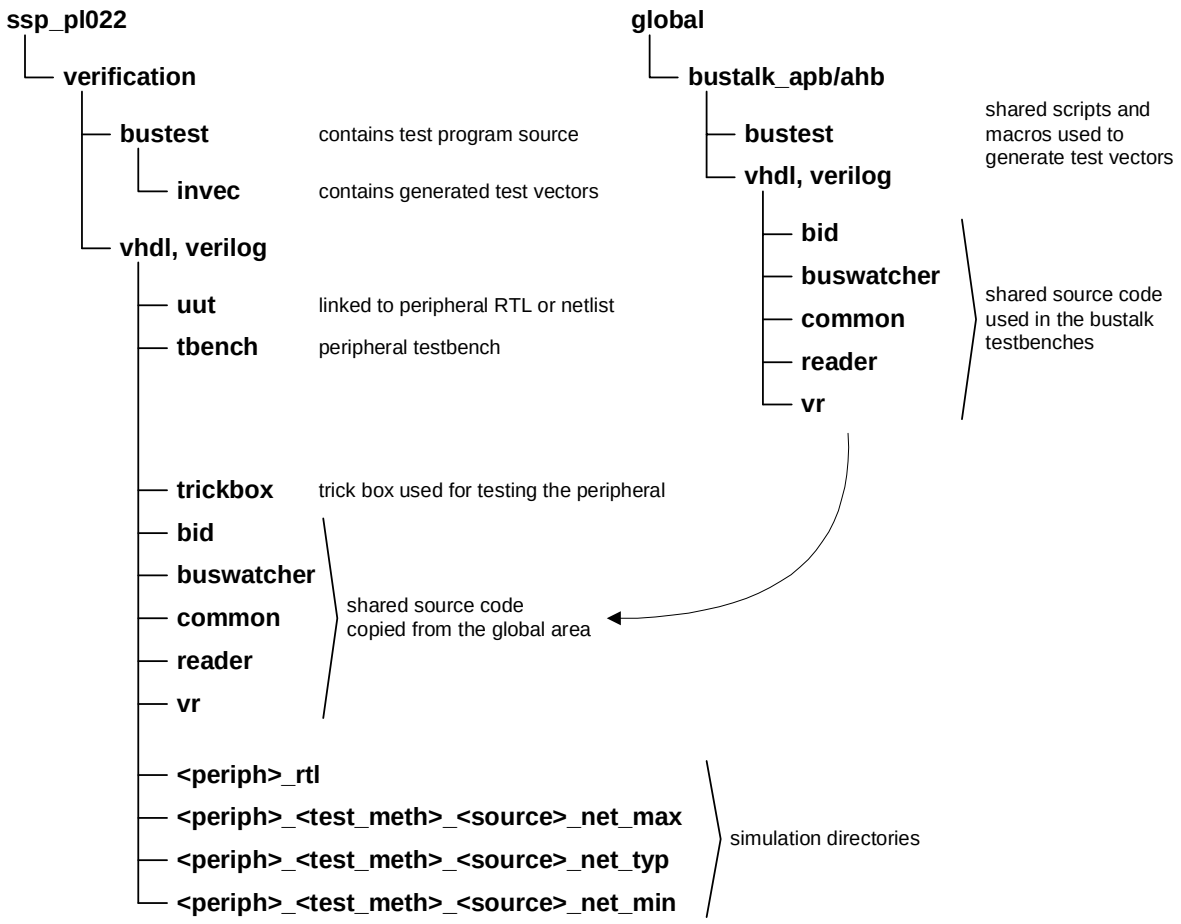


Figure 3-4. Generic Verification directory structure

The **uut** directory is linked either to the RTL source or the netlist directory depending on the simulation being performed.

3.4 Generic Integration Test directory structure

The integration test environment contains both Verilog and VHDL testbenches and the integration test. The testbenches are a minimal EASY system supporting TIC testing.

Figure 3-5 illustrates the generic integration test directory structure:

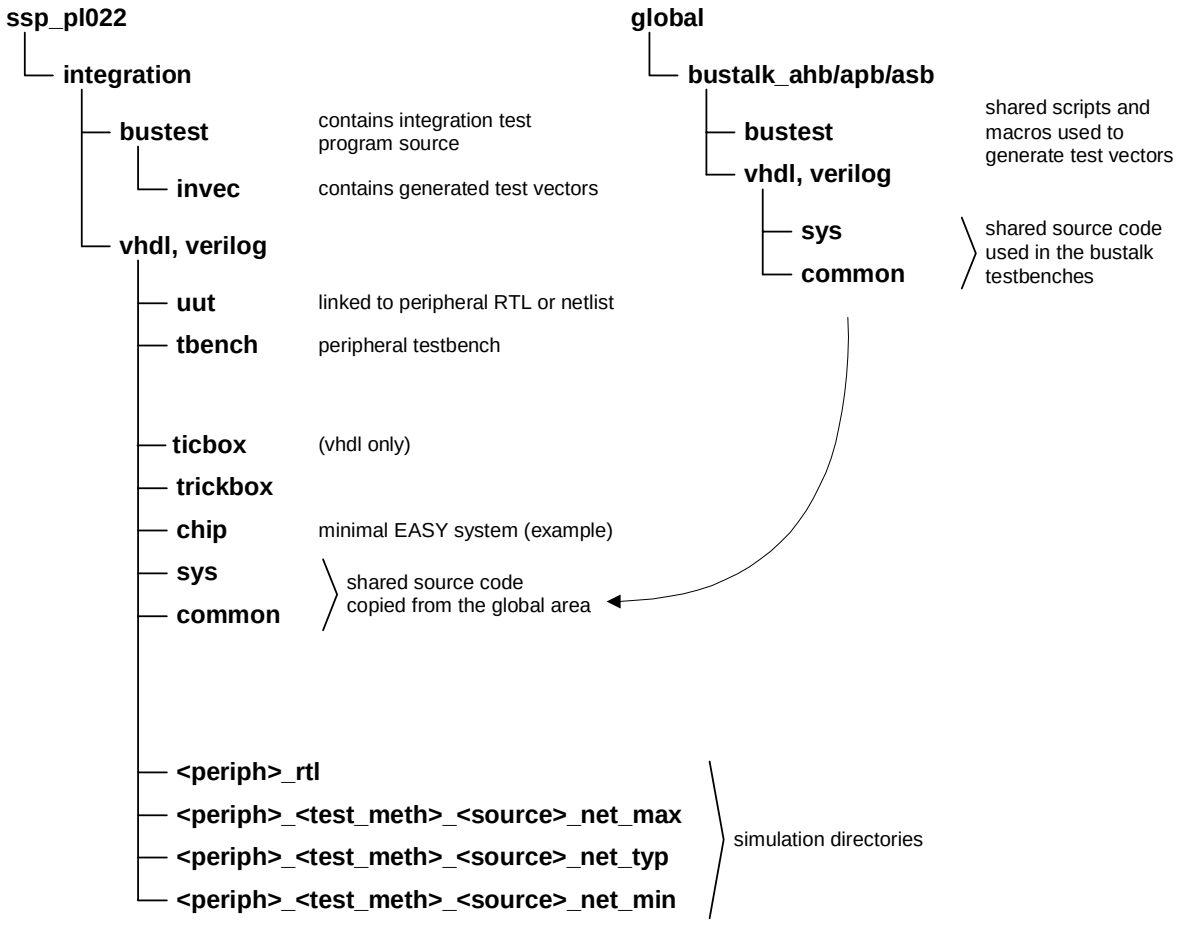


Figure 3-5. Generic TicTalk testbench directory structure

4 USAGE INSTRUCTIONS

4.1 Introduction

This PrimeCell peripheral has been designed in both synthesisable VHDL and Verilog Hardware Description Languages (HDL).

The peripheral is supplied with a standalone functional verification testbench using Bus Compliance (BusTalk) methodologies. Additionally, an integration testbench is supplied based on AMBA TicTalk for purposes of testing integration vectors.

4.2 Design tools used

The release note for each PrimeCell peripheral describes the simulation and synthesis tools used. Where different design tools are to be used, then amendments to the configuration, script and make files will be required.

The simulation examples below are illustrated using Model Technology Inc, ModelSim commands for both VHDL and Verilog designs, and Cadence LDV Verilog-XL for the Verilog designs. Amend these accordingly when other simulation tools are to be used.

VHDL and Verilog simulations have been performed with Model Technology Inc, ModelSim. The ModelSim simulator is an event-driven simulator that produces compiled environments for both VHDL and Verilog.

Verilog simulations have also been performed using Cadence LDV Verilog-XL. This simulator is an event-driven simulator which interprets the HDL at run-time.

Code coverage has been performed using the TransEDA Technology Limited Verification Navigator (VN-Cover) tool.

For further information, refer to the README files within the tbench directories for each testbench.

4.3 Reference Cell library

The Avant! CB25 (0.25 µm) standard cell library is supplied with each PrimeCell peripheral as a reference cell library.

This proprietary Avant! Passport standard cell library has been used during development of the peripheral, for synthesis and gate-level simulation with post-synthesis back-annotated timings in Standard Delay Format (SDF). For further information on this cell library, refer to the documentation supplied.

Use of a different technology library or design tools requires that script files and configuration files must be amended for the user's chosen technology library, simulation and synthesis tools.

For further information, particularly regarding amendment of synthesis script files to a different technology library, and integration of peripherals in an AMBA-based ASIC, refer to the Integration Manual of the peripheral concerned.

The availability of this reference cell library provides a means for the end user to replicate the synthesis and verification processes performed prior to delivery. Please note that this is a reference library only with no physical views, and users must provide their own target library for the development of their system.

The gates count and performance relate to the reference library only and it is believed that this library is representative of a typical vendor cell library. Synthesis to the customers target library should yield similar gate counts, but performance is vendor library specific.

Figure 4.1 illustrates how the cell library directory structure should be created relative to the PrimeCell peripherals. The top-level cb25 directory can be placed in the same directory level as the top-level of the PrimeCell peripherals, in order to run the gate-level simulations as defined in configuration files, shell scripts and usage instructions described later.

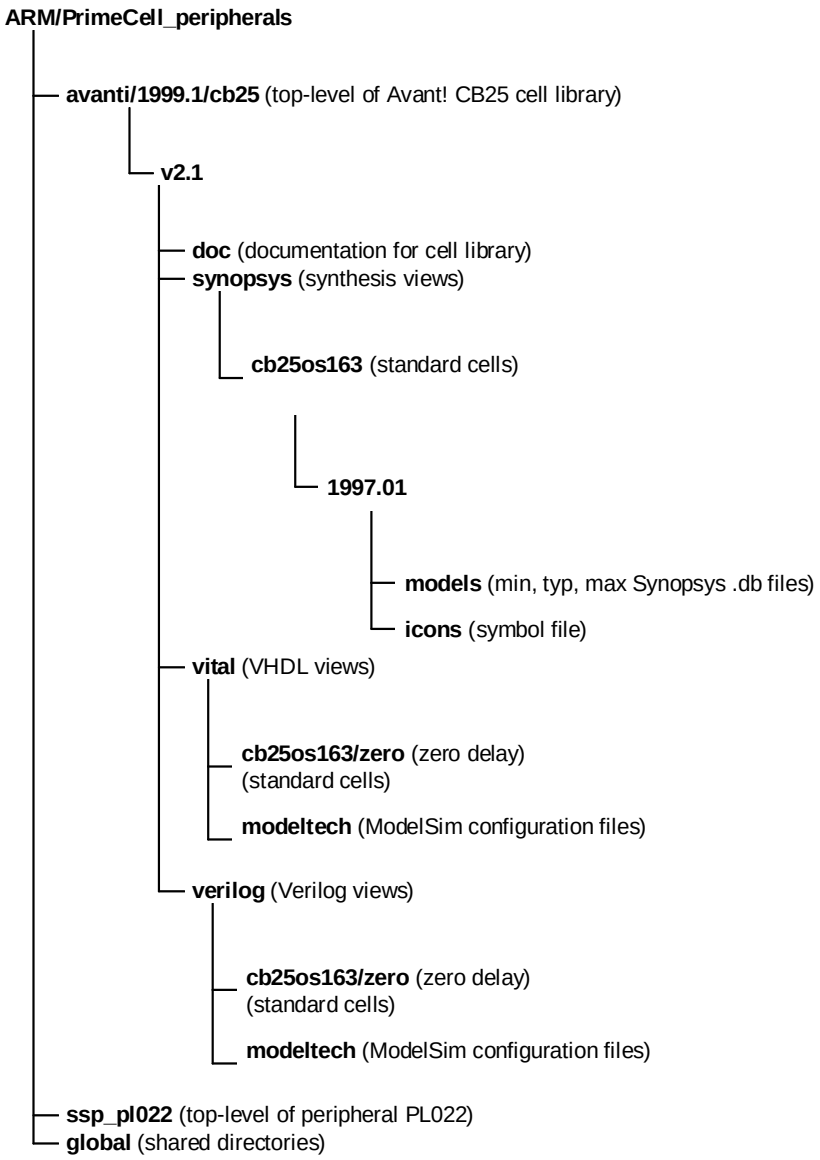


Figure 4.1 Avant! CB25 cell library directory structure

If the ModelSim simulator is being used, then create modeltech/ directories as in above structure for configuration files. Makefiles or compilation shell scripts also need to be created for the cell library models.

Create links as below to select zero (or maximum delay) models.

```
cd AMBA_peripherals/cb25/v2.1
ln –s vital/cb25os163_zero/ vhdl
ln –s vlog/cb25os163_zero/ verilog
```

4.4 Peripheral Development Environment

The peripheral development environment is designed to support a number of standard tools within a defined directory structure. To control how these tools are used a number of UNIX environment variables and makefiles are provided. This minimises the amount of user interaction required to support the tool chain.

To support the different requirements of the end user, a number of design parameters are supported. These are used to define which of the supported design flows is to be used.

Additionally, environmental parameters are supported to allow the end user to customise the peripheral development environment, allowing integration with their own development environment.

In principle, running a design tool (simulation, synthesis, …) consists of two steps:

1) setting up environment variables

2) using make tool to start a design tool

The make tool will then create necessary directory structure, setup all files required and start design tool.

4.4.1 Environment variables

UNIX environment variables are used to simplify the configuration of the directory paths, design tools and design flow options. A brief overview of the variables is given in ***Table 4-1***.

Variable	Value	Description
GLOBAL	<global_dir_path>	The path to the shared files.
PERIPH	<Peripheralname>	The peripheral name i.e. Ssp
SIMULATOR	modelsim, LDV	Defines which simulator is going to be used.
TEST_METH	noscan, scanready, scaninsert	Defines the scan methodology adopted during synthesis.
HDL_SOURCE	vhdl, verilog	The source code type – VHDL or Verilog.
HDL_NETL	vhdl, verilog	The netlist type – VHDL or Verilog.
TEST_ENV	BUSTALK, TICTALK	Defines the test environment - functional or integration test.
AMBA	APB, AHB	Defines the bus interconnection type and associated modules
LIB_VHDL	<vhdl_library_path>	The location of the VHDL cell library.
DIRS_VERILOG	<verilog_library_path>	The location of the Verilog cell library.
FILES_VERILOG	<verilog_udp_path>	The location of the Verilog UDP file.
LIB_SYNOP	<synopsys_library_path>	The location of the synopsys target library.

Table 4-1 Environment Variables

The peripheral's top-level makefile will overwrite the PERIPH variable with the correct name, but before running the make routine this variable must be declared within the UNIX environment. This enables integration of the PrimeCell peripheral makefile structure into a project (ASIC) makefile structure, where multiple PrimeCell peripherals might exist and each one of them requires the PERIPH variable to have a different value.

4.4.2 Makefile commands

The makefile structure consists of the peripheral specific makefiles and the shared makefiles (located in the global directory, which is referenced by the GLOBAL environment variable). The peripheral makefiles are used to move through the peripheral directory structure, setup the environment and then execute a shared makefile. The peripheral makefiles consist of the top-level makefile located in the <peripheral>_plxxx directory and the makefiles in the peripheral sub-directories.

The shared makefiles are located in the $GLOBAL directory, these makefiles are common to all peripherals in a system/ASIC and contain design tools specific commands. This enables an easy and quick modification to the makefile structure, if a design flow or a design tool needs to be changed.

A summary of the top-level makefile commands is given in ***Table 4-2***. Only the top-level makefile will be explained here, see the “README” and “makefile” files in the peripheral sub-directories for more details.

Make Argument	Commands	Description
all	cd $HDL_SOURCE make all	compiles RTL source code for simulation
default	cd verification make default	generates BusTalk test vector
rtl	cd verification make rtl	block level RTL simulation
cover	cd verification make cover	RTL code coverage
netlist	cd synopsys make netlist	synthesis
net_max	cd verification make net_max	maximum delay netlist simulation
net_typ	cd verification make net_typ	typical delay netlist simulation
net_min	cd verification make net_min	minimum delay netlist simulation
compare	cd chrysalis make netlist	RTL to netlist equivalence check
rtl_check	cd chrysalis make rtl	VHDL RTL to Verilog RTL equivalence check
clean	cd vhdl; make clean cd verilog; make clean cd verification; make clean cd integration; make clean cd synopsys; make clean cd chrysalis; make clean	removes work files in the sub-directories

Table 4-2 Makefile Commands

4.5 Running Design Tools

This chapter describes a step-by-step procedure for performing various tasks in the design flow.

4.5.1 Generating the Functional Test Vectors (BusTalk)

The BusTalk test compliance programs are written in 'C' and located in the verification/bustest directory. The generated BusTalk (.sim and .bif) test vector files are located in the verification/bustest/invec directory.

To generate the test vectors, first setup the following environment variables:

setenv GLOBAL	<global_dir_path>	
setenv PERIPH	<Peripheralname>	i.e. Ssp
setenv SIMULATOR	modelsim, LDV	

Then in the ssp_pl022 directory (peripheral's root dir) type:

make default

This will create two test vector files in the verification/bustest/invec directory, the infile.bif is used in the VHDL testbench and the bif.sim in the Verilog testbench.

The make default command creates the test vector that contains all test described in the Design Manual.

It is also possible to run each test in isolation as follows:

cd verification/bustest	
make <test_name>	e.g. make REG_TESTS

to generate a test vector that contains only that specific test. See the README and makefile files in the verification/bustest directory for more details.

4.5.2 Compiling the Peripheral RTL Code

Ensure that the relevant tools have been sourced.

To compile the peripheral RTL code, first set the environment variables:

setenv GLOBAL	<global_dir_path>	
setenv PERIPH	<Peripheralname>	- i.e. Ssp
setenv SIMULATOR	modelsim, LDV	- selects ModelSim or LDV simulator
setenv HDL_SOURCE	vhdl, verilog	- selects VDHL or Verilog RTL code
setenv TEST_ENV	BUSTALK	

Then, in the ssp_pl022 directory, type:

make all

to compile the peripheral RTL code in the correct order.

4.5.3 Running Functional Simulations

Ensure that the relevant tools have been sourced.

Before running the functional simulation, ensure that the functional test vectors are generated. To perform a functional simulation of the peripheral RTL, first set the environment variables:

setenv GLOBAL	<global_dir_path>	
setenv PERIPH	<Peripheralname>	- i.e. Ssp
setenv SIMULATOR	modelsim, LDV	- selects ModelSim or LDV simulator
setenv HDL_SOURCE	vhdl, verilog	- selects VDHL or Verilog RTL code
setenv TEST_ENV	BUSTALK	

Then, in the ssp_pl022 directory, type:

make rtl

to run the functional simulation of the peripheral RTL code. The simulation will terminate correctly with the following message to signal the completion of the test:

** Failure: End of test

(This message is a consequence of the simulator mechanism for termination)

There should be no errors reported during the simulation for a successful run, except during the initialisation period. It is advised that simulation transcript files always be checked for any errors or warnings during any subsequent simulations. The simulation log file will be written to the verification/$(HDL_SOURCE)/$(PERIPH)_rtl directory.

The simulation log file name depends on the simulator used, for ModelSim it is ‘transcript’ and for Cadence LDV Verilog-XL it is ‘verilog.log’.

The generated simulation log files are:

- verification/vhdl/$(PERIPH)_rtl/transcript	VHDL RTL, ModelSim simulator
- verification/verilog/$(PERIPH)_rtl/transcript	Verilog RTL, ModelSim simulator
- verification/verilog/$(PERIPH)_rtl/verilog.log	Verilog RTL, Cadence LDV Verilog-XL simulator

4.5.4 Running RTL Code Coverage

Ensure that the relevant tools have been sourced.

Before running the RTL code coverage simulation, ensure that the functional test vector is generated. To perform RTL code coverage, first set the environment variables:

setenv GLOBAL	<global_dir_path>	
setenv PERIPH	<Peripheralname>	- i.e. Ssp
setenv SIMULATOR	modelsim	
setenv HDL_SOURCE	vhdl/verilog	- selects VDHL or Verilog RTL code

```
setenv TEST_ENV        BUSTALK
```

Then, in the ssp_pl022 directory, type:

```
make cover
```

to run the code coverage of the peripheral RTL code. The code coverage history and summary files will be written to the verification/$(HDL_SOURCE)/$(PERIPH)_rtl directory.

The supplied code coverage summary files are:

- verification/vhdl/$(PERIPH)_rtl/vnavigator.summary VHDL RTL
- verification/verilog/$(PERIPH)_rtl/vnavigator.summary Verilog RTL

4.5.5 Running Synthesis

Ensure that the relevant tools have been sourced.

Synopsys Design Compiler is used for the peripheral synthesis, which targets the Avant! CB25 standard cell library.

The synopsys/scripts directory contains the peripheral specific synthesis scripts and the shared scripts are located in the $GLOBAL/synopsys_shared/scr_common directory. In order to change the target library or the clock frequency, only the shared scripts will need to be changed.

To run synthesis, first set the environment variables:

```
setenv GLOBAL       <global_dir_path>
setenv PERIPH       <Peripheralname>              - i.e. Ssp
setenv TEST_METH    noscan, scanready, scaninsert - selects the test method
setenv HDL_SOURCE   vhdl, verilog                 - selects VDHL or Verilog RTL code
setenv HDL_NETL     vhdl, verilog                 - selects VHDL or Verilog netlist
setenv LIB_SYNOP    <synopsys_lib_path>
```

To make use of the the synopsys cache:

Check that the ssp_pl022/global/synopsys_shared/synopsys_cache directory exists and is write enabled.

Then, in the ssp_pl022 directory, type:

```
make netlist
```

Synthesis log and report files will be written in the synopsys/log and synopsys/report directories, respectively.

The HDL_SOURCE variable selects the HDL source being compiled and the HDL_NETL variable controls the output format of the netlist.

The TEST_METH variable selects the test methodology used for the synthesis:

- noscan: - the peripheral will be synthesised without inserting a scan chain and scan enabled flip-flops
- scanready: - the peripheral will be synthesised with scan enabled flip-flops, but without inserting a scan chain
- scaninsert: - the peripheral will be synthesised with a scan chain inserted

4.5.5.1 scr_common directory

The scr_common directory contains the scripts that are shared by the PrimeCell peripherals. The files listed below are referenced in the peripherals synthesis script and must always be present.

- setup.scr – this file is used to define various Synopsys variables that are to be used during synthesis e.g. bus naming style.
- global.scr – this file defines global synthesis parameters which must be applied to all the PrimeCell peripherals e.g. set_max_area.
- amba_params.scr – these files define the timing parameter to be used for the various types of AMBA interfaces.
- ahb_slave.scr – these files define the timing constraints to be used for the various types of AMBA interfaces.
- library_<libraryname>.scr – this file contains everything to do with the cell library and the operating conditions. Using a single file to define all the library information makes it simple for the customer to switch the target library.
- name_rules.scr – the Synopsys naming rules, used to change the names of the cells, nets and ports in the design.

4.5.5.2 scripts directory

The scripts directory is specific to the peripheral and contains the synthesis command script and the synthesis constraints file that apply only to the peripheral. This directory contains the following files ;

- Ssp.cmd - this is the synthesis command script used for the peripheral. This command script references the scripts contained in this and scr_common directories.
- Ssp_scan.cmd – this is the command script used specifically for scan insertion. If scan insertion is not required this command script will not be used.
- Ssp.scr – this file defines the timing requirements for the peripheral's non-amba signals.
- Ssp_exceptions.scr – this file contains all the timing exceptions e.g. false and mutli-cycle paths.

4.5.5.3 log directory

The log directory is used to store the information generated about the synthesis run. Two files are created:

- $(PERIPH)_$(TEST_METH)_$(HDL_SOURCE)_$(HDL_NETL)_synth.log – this file is generated within the synthesis command script and is used to record the output from the checks performed on the design being synthesised i.e. "check_timing" and "check_test" both pre and post scan insertion.
- $(PERIPH)_$(TEST_METH)_$(HDL_SOURCE)_ $(HDL_NETL)_run.log – the synthesis run log is generated by piping the output of the "dc_shell" into this file.

All errors in these files must be fixed and the number of warning must be reduced to a minimum. The synthesis run log file might have following errors, which should be ignored:

> Error: You don't have a 'HDL-Compiler' license to remove.
>
> Error: You don't have a 'Test-Compiler' license to remove.

NOTE: When the "noscan" option is used the 'check_test' command isn't executed as it requires a TestCompile licence.

4.5.5.4 report directory

The report directory contains the reports generated from the synthesis run. The file name of the generated report varies depending on the parameters defined for the synthesis run i.e. the source code type and the test requirement. This allows different configuration to be synthesised without over-writing existing report files.

The reports generated by the synthesis scripts can be split into the a number of groups; violations report, timing reports, area report, constraint report and a scan chain report:

- **Violation report:** "$(PERIPH)_$(TEST_METH)_$(HDL_SOURCE)_$(HDL_NETL).vio

  The violation report contains a summary of all the constraint violations, the report is generated by the "report_constraint -verbose -all_violators" command.

  The only acceptable violation is a design area violation.

- **Timing reports:** "$(PERIPH)_$(TEST_METH)_$(HDL_SOURCE)_$(HDL_NETL).max/min

  The two timing reports are generated to enable the actual timing of the synthesised netlist to be examined.

  The max path report file (*.max) contains a timing report for the ten longest under worst case conditions and is generated using the "report_timing -delay max -path full -nworst 10" command.

  The min path report file (*.min) contains a timing report for the ten shortest paths under best case conditions and is generated using the "report_timing -delay min -path full -nworst 10" command.

- **Area report:** "$(PERIPH)_$(TEST_METH)_$(HDL_SOURCE)_$(HDL_NETL).area

  The area report contains the area for each of the modules in the design.

- **Latch report:** "$(PERIPH)_$(TEST_METH)_$(HDL_SOURCE)_$(HDL_NETL).latch

  The latch report contains information of all latches and timing loops found in the design, the report is generated using the "all_registers -level_sensitive" and "report_timing –loops" commands

- **Constraint reports:** "$(PERIPH)_$(TEST_METH)_$(HDL_SOURCE)_$(HDL_NETL).ports/constr

  For further information the constraints used during synthesis are reported in the following two files.

  The ports report contains (*.ports) information about the constraints placed on each of the peripheral's top level ports and is generated using the "report_port –verbose" command.

  The constraint report (*.constr) contains general information about the netlist that has been created, this report is generated using the "report_design", "report_constraint", "report_clock" and "report_attribute –design" commands.

- **Scan chain report:** "$(PERIPH)_scaninsert_$(HDL_SOURCE)_$(HDL_NETL).scan

  The scan chain report is created by the "Ssp_scan.cmd" script and is only generated when scan insertion has been performed. The report details the expected fault coverage, the scan configuration and the scan chain order using the "report_test –coverage", "report_test –configuration" and "report_test –scan_path" commands respectively.

4.5.6 Running Netlist Simulations

Ensure that the relevant tools have been sourced.

Before running the netlist simulation, ensure that the functional test vector is generated. The vhdl or verilog cell library must also be compiled and performing a “make clean” in the respective area can do this:

cd avanti/1999.1/cb25/v2.1/vital/cb25os163/zero/ then, type> make clean

cd avanti/1999.1/cb25/v2.1/verilog/cb25os163/zero/ then, type> make clean

To run the netlist simulation, set the environment variables:

setenv GLOBAL	<global_dir_path>	
setenv PERIPH	<Peripheralname>	- i.e. Ssp
setenv SIMULATOR	modelsim, LDV	
setenv TEST_METH	noscan, scanready, scaninsert	- selects the test method
setenv HDL_SOURCE	vhdl, verilog	- selects VDHL or Verilog RTL code
setenv HDL_NETL	vhdl, verilog	- selects VHDL or Verilog netlist
setenv DIRS_VERILOG	<verilog_cell_lib_path>	- set if HDL_NETL = verilog
setenv LIB_VHDL	<vhdl_cell_lib_path>	- set if HDL_NETL = vhdl

Note that the TEST_METH, HDL_SOURCE and HDL_NETL variables should have the same values for the peripheral synthesis and the netlist simulation.

Then, in the ssp_pl022 directory, type:

make net_max	- to run ‘worst case’ netlist simulation
make net_typ	- to run ‘typical case’ netlist simulation
make net_min	- to run ‘best case’ netlist simulation

The simulation will terminate correctly with the following message to signal the completion of the test:

```
** Failure: End of test
```

There should be no errors reported during the simulation for a successful run, except during the initialisation period. It is advised that simulation transcript files are always checked for any errors or warnings during any subsequent simulations. The simulation log file will be written to one of the following directories:

verification/$HDL_NETL/<periph._<test_meth>_<hdl_source>_net_max - worst case simulation log

verification/$HDL_NETL/<periph>_<test_meth>_<hdl_source>_net_typ - typical simulation case log

verification/$HDL_NETL/<periph>_<test_meth>_<hdl_source>_net_min - best simulation case log

The simulation log file name depends on the simulator used, for ModelSim it is ‘transcript’ and for Cadence LDV Verilog-XL it is ‘verilog.log’.The supplied simulation log files are:

- verification/verilog/<periph>_noscan_vhdl_net_min/<periph>_noscan_vhdl_net_min_modelsim.log
- verification/verilog/<periph>_scanready_verilog_net_typ/<periph>_scanready_verilog_net_typ_LDV.log
- verification/vhdl/<periph>_scaninsert_vhdl_net_max/<periph>_scaninsert_vhdl_net_max_modelsim.log

4.5.7 Running VHDL to Verilog RTL Equivalence Check

To run the VHDL to Verilog RTL equivalence check, first set the environment variables:

```
setenv  GLOBAL      <global_dir_path>
setenv  PERIPH      <Peripheralname>      - i.e. Ssp
```

Then, in the ssp_pl022 directory, type:

```
make    rtl_check
```

to run the equivalence check of the peripheral RTL code. The make tool will first compile VHDL and Verilog RTL to the Chrysalis internal format and then it will perform RTL to RTL comparison.

The compilation log files are Ssp_vhdl_rtl.log_build and Ssp_verilog_rtl.log_build in the chrysalis/buildrtl directory, the log files should not contain any warning, error or fatal messages. However, there might be notes regarding the unconnected and undriven scan chain signals, for example:

```
Note    406:1 Signal 100  - SCANENABLE is statically constrained to 0
Note    277:5 Signal 101  - SCANINPCLK (primary input) drives unconnected logic and has been removed from model
Note    277:5 Signal 102  - SCANINSSPCLK (primary input) drives unconnected logic and has been removed from model
…
Note    277:2     signal 149  - SCANOUTPCLK is undriven
Note    277:2     signal 150  - SCANOUTSSPCLK is undriven
…
************************ End File Summary ************************************
   Cpu Time  00:00:35          Total Dynamic Memory = 36 Mbyte
   9  Notes, 0  Warnings, 0  Errors, 0  Fatal Msgs, 0  Restart Msgs
```

The comparison report file is chrysalis/reports/<periph>_vhdlTOverilog.log_verify, the report file should not contain any warnings, errors or fatal messages. The RTL to RTL comparison should report zero unverified key points.

Note that there might be a number of stuck primary outputs in the report file, these are scan chain outputs and should be ignored.

4.5.8 Running RTL to Netlist Equivalence Check

To run the RTL to netlist equivalence check, first set the environment variables:

setenv GLOBAL	<global_dir_path>	
setenv PERIPH	<Peripheralname>	- i.e. Ssp
setenv TEST_METH	noscan, scanready, scaninsert	- selects the test method
setenv HDL_SOURCE	vhdl, verilog	- selects VDHL or Verilog RTL code
setenv HDL_NETL	vhdl, verilog	- selects VHDL or Verilog netlist
setenv DIRS_VERILOG	<verilog_cell_lib_path>	
setenv LIB_VHDL	<vhdl_cell_lib_path>	

Note that the TEST_METH, HDL_SOURCE and HDL_NETL variables should have the same values for the peripheral synthesis and the RTL to netlist equivalence check.

Then, in the ssp_pl022 directory, type:

```
make compare
```

to run the equivalence check of the peripheral RTL source code and netlist. The make tool will first compile the RTL and the netlist to the Chrysalis internal format and then it will perform RTL to netlist comparison.

The compilation log files are chrysalis/buildrtl /$(PERIPH)_$(HDL_SOURCE)_rtl.log_build for the RTL compilation, and chrysalis/buildnet /$(PERIPH)_$(TEST_METH)_$(HDL_SOURCE)_$(HDL_NETL)_net.log_build for the netlist compilation. The log files should not contain any warning, error or fatal messages. However, there might be notes regarding the unconnected and undriven scan chain signals, see the previous section for more details. Also, if the test method is scan insert, the netlist log file will not contain notes about undriven scan chain signal (total number of notes will be 5).

The comparison report file is located in the chrysalis/reports directory, the file name is $(PERIPH)_$(HDL_SOURCE)_rtlTO$(PERIPH)_$(TEST_METH)_$(HDL_SOURCE)_$(HDL_NETL)_net.log_verify.

The report file should not contain any warnings, errors or fatal messages. The RTL to netlist comparison should report zero unverified key points.

Note that there might be 4 stuck primary outputs in the report file, these are scan chain outputs and should be ignored.

Note that if the test method is scan insert, the report file might contain warnings that are result of the scan chain(s) insertion. The report file should be carefully examined to ensure that the warnings are caused by the scan chain insertion. Warning example:

```
Warning 272:135 Output Name Mismatch - 1  Primary Outputs differ by name
Warning 272:138 Output Mismatch - 3  Primary Outputs proven to non-outputs
Warning 272:140 Extra Stuck Output - 4  extra Stuck Primary Outputs
```

4.5.9 Generating the Integration Test Vectors

To generate the integration test vectors, first set the environment variables:

setenv GLOBAL <global_dir_path>

setenv PERIPH <Peripheralname> - i.e. Ssp

setenv SIMULATOR modelsim, LDV

Then, in the ssp_pl022 directory type:

cd integration

make default

This will create the integration test vectors in the ssp_pl022/integration/bustest/invec directory.

Note that the integration tests are only an example and that they should be modified when the peripheral is integrated into the system. Please see the peripheral's integration manual for more details.

4.5.10 Running the Integration Test RTL Simulation

To perform the integration test simulation, first set the environment variables:

setenv GLOBAL <global_dir_path>

setenv PERIPH <Peripheralname> -i.e. Ssp

setenv SIMULATOR modelsim, LDV

setenv HDL_SOURCE vhdl, verilog

Then, in the ssp_pl022 directory type:

cd integration

make rtl

The simulation will terminate correctly with the following message to signal the completion of the test:

```
** Failure: End of test
```

There should be no errors reported during the simulation for a successful run, except during the initialisation period. It is advised that simulation transcript files are always checked for any errors or warnings during any subsequent simulations. The simulation log file will be written to the /integration/$(HDL_SOURCE)/$(PERIPH)_rtl directory.

Note that the integration testbench is only an example of an ASIC system and that the integration test should be performed on the actual system. Please see the peripheral's design manual for more details on the integration testbench.

Following error and warning messages may be found in the simulation log files during the initialisation period and are not significant:

From VHDL simulation:

** Warning: There is an 'U'|'X'|'W'|'Z'|'-' in an arithmetic operand, the result will be 'X'(es)!

** Warning: There is an 'U'|'X'|'W'|'Z'|'-' in an arithmetic operand!

From Verilog simulation:

```
# WARNING: (time:        0) *** Buswatcher set to use Decode Cycles ***
# ERROR: (time:        0) BERROR driven during BCLK high
# ERROR: (time:        0) BLAST driven during BCLK high
# ERROR: (time:        0) BWAIT driven during BCLK high
```

4.5.11 Running the Integration Test Code Coverage Simulation

To perform the integration test code coverage, first set the environment variables:

setenv GLOBAL	<global_dir_path>	
setenv PERIPH	<Peripheralname>	- i.e. Ssp
setenv SIMULATOR	modelsim, LDV	
setenv HDL_SOURCE	vhdl, verilog	

Then, in the ssp_pl022 directory type:

```
cd integration
make cover
```

This code coverage summary file will be written to the integration/$(HDL_SOURCE)/$(PERIPH)_rtl directory. The aim of the integration test code coverage is to achieve 100% toggle of the peripherals input and output signal (excluding the scan chain signals). This test is not mandatory.

The simulation log file name depends on the simulator used, for ModelSim it is ‘transcript’ and for Cadence LDV Verilog-XL it is ‘verilog.log’.

Note that the integration testbench is only an example of an ASIC system and that the integration test should be performed on the actual system. Please see the peripheral’s design manual for more details on the integration testbench.

4.5.12 Running the Integration Test Netlist Simulation

To perform the integration test code coverage, first set the environment variables:

setenv GLOBAL	<global_dir_path>	
setenv PERIPH	<Peripheralname>	- i.e. Ssp
setenv SIMULATOR	modelsim, LDV	
setenv TEST_METH	noscan, scanready, scaninsert	- selects the test method
setenv HDL_SOURCE	vhdl, verilog	- selects VDHL or Verilog RTL code
setenv HDL_NETL	vhdl, verilog	- selects VHDL or Verilog netlist
setenv DIRS_VERILOG	<verilog_cell_lib_path>	- set if HDL_NETL = verilog
setenv LIB_VHDL	<vhdl_cell_lib_path>	- set if HDL_NETL = vhdl

Note that the TEST_METH, HDL_SOURCE and HDL_NETL variables should have the same values for the peripheral synthesis and the netlist simulation.

Then, in the ssp_pl022/integration directory, type:

make net_max	to run ‘worst case’ netlist simulation
make net_typ	to run ‘typical case’ netlist simulation
make net_min	to run ‘best case’ netlist simulation

The simulation will terminate correctly with the following message to signal the completion of the test:

** Failure: End of test

There should be no errors reported during the simulation for a successful run, except during the initialisation period. It is advised that simulation transcript files are always checked for any errors or warnings during any subsequent simulations. The simulation log file will be written to one of the following directories:

integration/$(HDL_NETL)/$(PERIPH)_$(TEST_METH)_$(HDL_SOURCE)_net_max - worst case simulation log

integration/$(HDL_NETL)/$(PERIPH)_$(TEST_METH)_$(HDL_SOURCE)_net_typ - typical simulation case log

integration/$(HDL_NETL/)$(PERIPH)_$(TEST_METH)_$(HDL_SOURCE)_net_min - best simulation case log

The simulation log file name depends on the simulator used, for ModelSim it is ‘transcript’ and for Cadence LDV Verilog-XL it is ‘verilog.log’.

Note that the integration testbench is only an example of an ASIC system and that the integration test should be performed on the actual system. Please see the peripheral’s design manual for more details on the integration testbench.